Universidad Nacional del Comahue

Centro Regional Zona Atlántica

Tecnicatura Superior en Administración de Sistemas y Software Libre

Automatización y Scripting

Trabajo Práctico N.º 4

Automatización Avanzada: Contenedores, Ansible y CI/CD

Alumno: Rolando Cobis
Legajo: CURZA-9389
DNI: 96.495.580
Docente: Ramiro García Poggi
Fecha de entrega: 5 de noviembre de 2026

Índice

Enunciado del Trabajo Práctico 2
Descripción del Caso Práctico 2
Tarea 1: Contenedores y Orquestación (Docker & Docker Compose) 2
Tarea 2: Gestión de Configuración Automática con Ansible 2
Tarea 3: Integración Continua (CI/CD) con GitHub Actions 3
Listado de Entregables: 3
Hipótesis, supuestos y aclaraciones 4
Tarea 1: Contenedores y Orquestación (Docker & Docker Compose) 5
Reporte en tiempo real 6
Tarea 2: Gestión de Configuración Automática con Ansible 7
Handler de Nginx 8
Tarea 3: Integración Continua (CI/CD) con GitHub Actions 9

Enunciado del Trabajo Práctico

Objetivo:
Implementar configuraciones modernas de infraestructura, empaquetar aplicaciones en contenedores ligeros con Docker/Podman, orquestar múltiples servicios utilizando Docker Compose, administrar y aprovisionar servidores de forma declarativa e idempotente usando Ansible Playbooks, y crear un flujo de integración continua (CI/CD) con GitHub Actions.
Recursos:
- Bibliografía de la cátedra y de asignaturas anteriores
- Plataforma PEDCO
- PC con Ubuntu o un Sistema operativo que soporte bash (puede hacerlo como alternativa con Alpine y sh, el script no será igual probablemente)
Práctica:

Descripción del Caso Práctico
La dirección de TIC de la universidad CURZA requiere implementar un servicio centralizado de análisis de tráfico web para los servidores del campus. El equipo docente ha decidido utilizar GoAccess (un analizador de logs interactivo) junto a servidores web Nginx. Usted debe automatizar el despliegue del software mediante contenedores y Ansible, y a su vez configurar validaciones automatizadas cada vez que se actualice la configuración de la red de laboratorios.
Requisitos Mínimos y Tareas Prácticas

Tarea 1: Contenedores y Orquestación (Docker & Docker Compose)
Dockerfile Personalizado:
Escriba un archivo `docker/Dockerfile` que use una imagen base de Alpine Linux e instale GoAccess.
El contenedor debe configurarse para leer los logs desde la ruta `/srv/logs/access.log`.
Agregue un usuario no-root dentro del contenedor por seguridad.
Orquestación Multi-Servicio (`docker-compose.yml`):
Diseñe un archivo `docker/docker-compose.yml` que defina dos servicios coordinados:
web_server: Levanta un servidor web Nginx utilizando la imagen oficial de Alpine. Debe exponer el puerto `[8000 + Últimos 2 dígitos del legajo]` del host hacia el puerto `80` del contenedor.
log_analyzer: Levanta su contenedor de GoAccess a partir de su Dockerfile.
Configure un volumen compartido para que el servidor web escriba sus logs de acceso (`access.log`) y GoAccess los lea e interprete en tiempo real, generando un archivo HTML estático (`report.html`).
Exponga el reporte interactivo de GoAccess en el puerto `[9000 + Últimos 2 dígitos del legajo]` para que el docente pueda ver los gráficos desde su navegador. GoAccess corre en modo tiempo real escribiendo report.html en un volumen compartido, y Nginx monta ese volumen sirviéndolo en el puerto 9000+XX (o en una ruta /report.html).

Tarea 2: Gestión de Configuración Automática con Ansible
Automatice el aprovisionamiento y despliegue del entorno.
Inventario (`ansible/hosts.ini`):
Defina un grupo de servidores llamado `[lab_systems]` que apunte a `localhost` (para pruebas locales).
Configure las variables necesarias para la conexión SSH local y la elevación de privilegios (`become: yes`).
Playbook de Aprovisionamiento (`ansible/playbook.yml`):
Escriba un playbook que automatice las siguientes tareas:
Instalar Docker y sus dependencias en el sistema de destino.
Copiar los archivos de configuración de Nginx y GoAccess al servidor destino.
Asegurar que el servicio de Docker esté iniciado y habilitado.

Levantar los contenedores de Docker Compose utilizando el módulo de Ansible `community.docker.docker_compose_v2` o ejecutando comandos mediante shell.
Handler: Defina un handler que reinicie el servidor Nginx si se modifica el archivo de configuración.

Tarea 3: Integración Continua (CI/CD) con GitHub Actions

Asegure que cualquier cambio de configuración en su infraestructura sea testeado antes de subirse.
Cree un script simple de pruebas en Python `tests/test_config.py` que valide que el archivo `docker--compose.yml` sea un archivo YAML sintácticamente correcto.
Cree un pipeline de integración continua configurando un archivo en la ruta `.github/workflows/ci--pipeline.yml`.
Requisitos del Workflow:
Debe dispararse ante eventos de `push` o `pull_request` sobre la rama `main`.
Debe utilizar un runner de Ubuntu moderno (`ubuntu-latest`).
Debe realizar los siguientes pasos (steps):
Clonar el código del repositorio (`actions/checkout`).
Configurar el entorno de Python 3.x (`actions/setup-python`).
Instalar dependencias necesarias (ej. `pyyaml` para validar el YAML).
Ejecutar las pruebas de Python utilizando `python -m unittest tests/test_config.py`.

Listado de Entregables:

Documento de Informe en formato PDF con: carátula (debe incluir el nombre e identificación del alumno), índice, copia del enunciado del problema a resolver y resolución de los puntos. Incluya todas las hipótesis, supuestos y aclaraciones que considere importante realizar.
Código completo en un repositorio de Github: deberá estar correctamente indentado y comentado para su sencilla comprensión.
El trabajo debe entregarse con la siguiente estructura de carpetas:

```
tp4-[legajo]/
├── README.md (Contiene explicaciones de puertos, links a capturas y diagramas)
├── .github/
│ └── workflows/
│ └── ci-pipeline.yml
├── docker/
│ ├── Dockerfile
│ ├── docker-compose.yml
│ ├── nginx.conf
│ └── goaccess.conf
├── ansible/
│ ├── hosts.ini
│ └── playbook.yml
└── tests/
└── test_config.py
```

README.md: Incluir capturas de pantalla de la terminal mostrando la ejecución exitosa de los scripts.
Commits: Debe haber un historial de al menos 3 commits con mensajes claros. Se penalizarán las entregas con un único push final.

Hipótesis, supuestos y aclaraciones

Usé la VM Ubuntu Server 24.04.5 de los prácticos anteriores. Ejecuté Ansible dentro de esa VM y configuré el inventario para conectar por SSH a `127.0.0.1:4089` como `rolando`. Mantuve ese puerto porque ya estaba configurado en Administración de Servicios.

Calculé 8089 y 9089 con los dos últimos dígitos del legajo. Abrí un túnel SSH desde la laptop para usar `localhost` en el navegador y en la dirección del WebSocket. Para acceso directo por IP dejé la variable `report_host` en el playbook.

Instalé Ansible como preparación del controlador y usé el playbook para instalar Docker y Compose. Probé con una autorización temporal de sudo sin contraseña; para repetir el despliegue sin ella, el comando admite `--ask-become-pass`. Las claves privadas quedaron fuera del repositorio.

Al arrancar encontré el reloj de la VM atrasado. Reinicié chrony y esperé a comprobar la sincronización antes de instalar paquetes. La primera configuración de Nginx duplicaba el formato incorporado `combined`; quité esa declaración y validé con `nginx -t`.

Guardé las salidas de terminal en archivos `.txt` y generé los PNG a partir de esos registros. Para el reporte de GoAccess tomé una captura real del navegador.

```
$ hostname
rolando-cobis-9389

$ date -Is
2026-11-05T17:33:55-03:00

$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.5 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo

$ sudo -n docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu3~24.04.2

$ sudo -n docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1~24.04.1

$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/rolando/.ansible/plugins/modules',
'/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/rolando/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Aug 31 2026, 10:18:26) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

Figura 1: Entorno y versiones usados en las pruebas.

Tarea 1: Contenedores y Orquestación (Docker & Docker Compose)

Construí la imagen de GoAccess con Alpine 3.22 y creé el usuario `analyzer` con UID 10001. Configuré la lectura de `/srv/logs/access.log` y la salida HTML en `/srv/reports/report.html`.

Usé dos volúmenes: `access_logs` para los registros y `reports` para el HTML. Nginx escribió los logs y GoAccess los leyó con un montaje de solo lectura. En el volumen del reporte invertí los permisos: GoAccess escribió y Nginx solo leyó.

Acceso	Host	Contenedor
Sitio Nginx	8089	80
Reporte y WebSocket	9089	90
GoAccess, solo red interna	Sin publicar	7890

Hice que GoAccess esperara al chequeo HTTP exitoso de Nginx. Esa petición generó la primera línea del log antes de iniciar el análisis; también quedó contabilizada en el reporte.

```
$ sudo -n docker ps --format "table {{.Names}}\t{{.Status}}"
NAMES                     STATUS
tp4-9389-log_analyzer-1   Up About a minute
tp4-9389-web_server-1     Up 31 seconds (healthy)

$ sudo -n docker port tp4-9389-web_server-1
80/tcp -> 0.0.0.0:8089
80/tcp -> [::]:8089
90/tcp -> 0.0.0.0:9089
90/tcp -> [::]:9089

$ sudo -n docker exec tp4-9389-log_analyzer-1 id
uid=10001(analyzer) gid=10001(analyzer) groups=10001(analyzer)

$ sudo -n docker inspect tp4-9389-log_analyzer-1 --format "{{range .Mounts}}{{.Destination}}
escritura={{.RW}}{{println}}{{end}}"
/srv/reports escritura=true
/etc/goaccess/goaccess.conf escritura=false
/srv/logs escritura=false


$ sudo -n docker exec tp4-9389-web_server-1 nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Figura 2: Servicios activos, usuario sin root y permisos de los volúmenes.

```
$ curl -s -o /dev/null -w 'Sitio HTTP %{http_code}\n' http://127.0.0.1:8089/
Sitio HTTP 200

$ curl -s -o /dev/null -w 'Reporte HTTP %{http_code}\n' http://127.0.0.1:9089/report.html
Reporte HTTP 200

$ sudo -n docker exec tp4-9389-log_analyzer-1 ls -l /srv/logs/access.log /srv/reports/report.html
-rw-r--r--    1 root     root          3164 Nov  5 20:33 /srv/logs/access.log
-rw-r--r--    1 analyzer analyzer    726704 Nov  5 20:32 /srv/reports/report.html
```

Figura 3: Sitio y reporte disponibles por HTTP; archivos compartidos.

Reporte en tiempo real

Serví el HTML por Nginx y configuré `/ws` como proxy hacia el puerto interno 7890 de GoAccess. Generé seis peticiones a rutas inexistentes y comprobé que el reporte mostró los seis 404. El navegador recibió mensajes nuevos por WebSocket sin recargar.

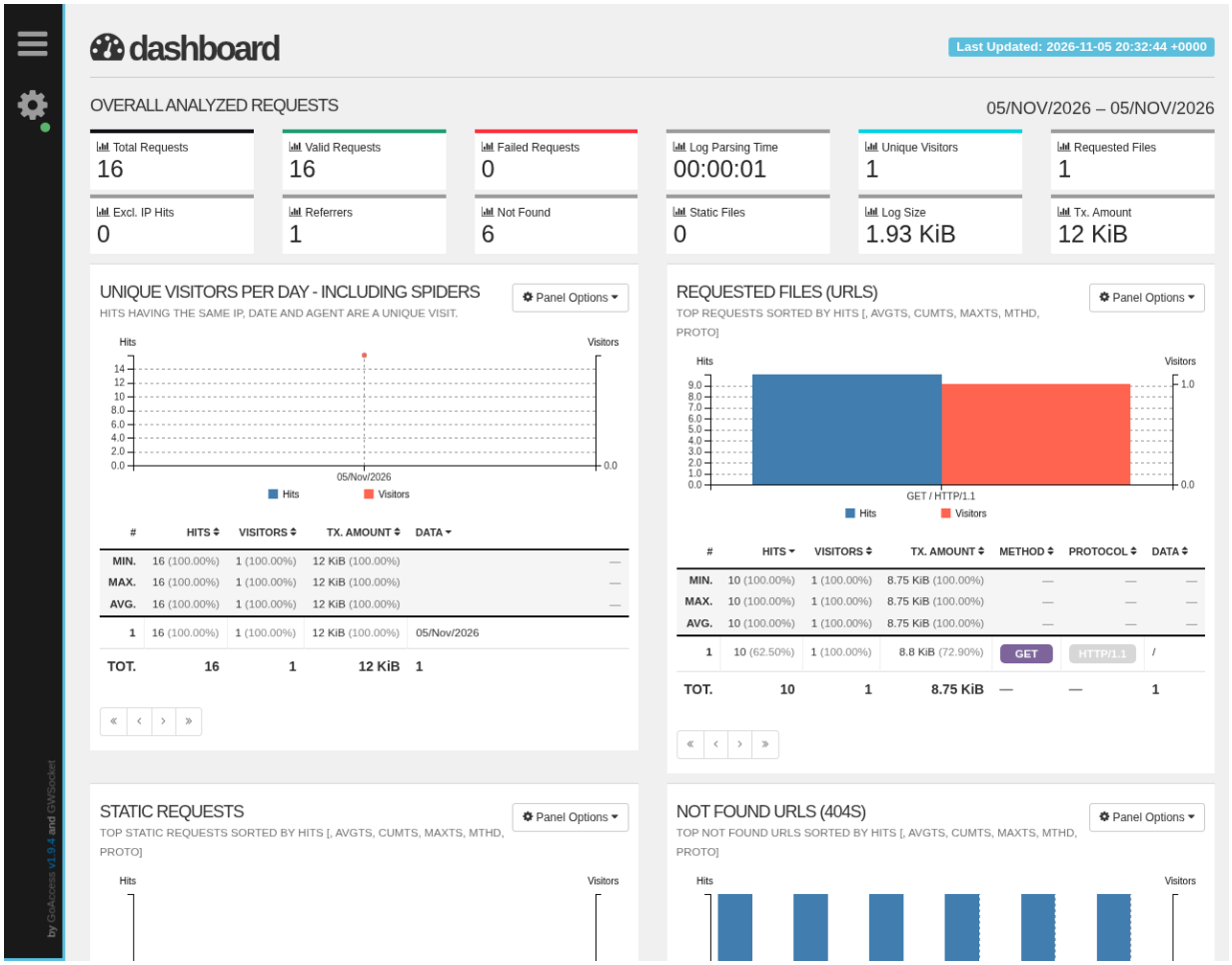


Figura 4: Reporte de GoAccess en el navegador después de generar tráfico.

```
URL: http://localhost:9089/report.html
HTTP: 200
WebSocket: ws://localhost:9089/ws
Mensajes antes de generar tráfico: 1
Mensajes después: 3
Generé seis peticiones a rutas /prueba-tp4-N (404).
El navegador recibió datos nuevos sin recargar.
```

Figura 5: Comprobación de la actualización sin recargar la página.

Tarea 2: Gestión de Configuración Automática con Ansible

Definí el grupo `lab_systems` en `ansible/hosts.ini` y habilité `become` para instalar paquetes y administrar Docker. Copié las configuraciones a `/opt/tp4-9389` y levanté los servicios con `community.docker.docker_compose_v2`.

Ejecuté el playbook desde la raíz del repositorio:

```
ansible-playbook -i ansible/hosts.ini ansible/playbook.yml
```

Después de corregir Nginx, completé el despliegue. Repetí el mismo comando sin modificar archivos: terminó con siete tareas correctas, cero cambios y cero errores.

```
$ ansible-playbook -i ansible/hosts.ini ansible/playbook.yml

PLAY [Desplegar el analizador de tráfico del campus] ***************************

TASK [Gathering Facts] *********************************************************
ok: [localhost]

TASK [Instalar Docker y Compose] ***********************************************
ok: [localhost]

TASK [Habilitar e iniciar Docker] **********************************************
ok: [localhost]

TASK [Crear el directorio de despliegue] ***************************************
ok: [localhost]

TASK [Copiar Dockerfile, Compose y configuración de GoAccess] ******************
ok: [localhost] => (item=Dockerfile)
ok: [localhost] => (item=docker-compose.yml)
ok: [localhost] => (item=goaccess.conf)

TASK [Copiar la configuración de Nginx] ****************************************
ok: [localhost]

TASK [Levantar los dos servicios] **********************************************
ok: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=7    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0
```

Figura 6: Segunda ejecución: changed=0 y failed=0.

Handler de Nginx

Agregué un comentario de prueba a `nginx.conf` y repetí el playbook. La copia del archivo notificó al handler `Reiniciar Nginx`. Comparé la hora de inicio del contenedor antes y después y confirmé el reinicio. Luego restauré el archivo original y apliqué nuevamente la configuración.

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
Inicio anterior: 2026-11-05T20:31:52.935729661Z

PLAY [Desplegar el analizador de tráfico del campus] **************************

TASK [Gathering Facts] *********************************************************
ok: [localhost]

TASK [Instalar Docker y Compose] ***********************************************
ok: [localhost]

TASK [Habilitar e iniciar Docker] **********************************************
ok: [localhost]

TASK [Crear el directorio de despliegue] ***************************************
ok: [localhost]

TASK [Copiar Dockerfile, Compose y configuración de GoAccess] ******************
ok: [localhost] => (item=Dockerfile)
ok: [localhost] => (item=docker-compose.yml)
ok: [localhost] => (item=goaccess.conf)

TASK [Copiar la configuración de Nginx] ****************************************
changed: [localhost]

TASK [Levantar los dos servicios] **********************************************
ok: [localhost]

RUNNING HANDLER [Reiniciar Nginx] **********************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost                  : ok=8    changed=2    unreachable=0    failed=0    skipped=0    rescued=0
ignored=0

Inicio posterior: 2026-11-05T20:33:26.266097269Z
```

Figura 7: Cambio de configuración y ejecución del handler.

Tarea 3: Integración Continua (CI/CD) con GitHub Actions

Configuré el workflow para los eventos `push` y `pull_request` sobre `main`, con un runner `ubuntu-latest`. Definí los pasos para preparar Python 3.12, instalar PyYAML y ejecutar la prueba pedida por la consigna.

```yaml
name: Validar configuración
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  config:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'
      - name: Instalar PyYAML
        run: python -m pip install pyyaml
      - name: Validar Compose
        run: python -m unittest tests/test_config.py
```

En `tests/test_config.py` cargué `docker/docker-compose.yml` con `yaml.safe_load` y comprobé la presencia de los dos servicios. Probé además un YAML mal formado: el test falló con `ParserError`. Restauré el archivo y volvió a pasar.

```
$ python -m unittest tests/test_config.py
.
----------------------------------------------------------------------
Ran 1 test in 0.003s

OK

Control negativo: YAML mal formado rechazado con ParserError.
Restauré el archivo y repetí la prueba: OK.
```

Figura 8: Prueba local y control negativo del parser.

Esta prueba verificó la sintaxis y los nombres de los servicios. Comprobé el funcionamiento del despliegue por separado, con las pruebas HTTP, de permisos y del WebSocket mostradas en la Tarea 1.